Interlude: Scope and Drop as Continuation Closure

Opening Question

When a value leaves scope, what exactly ends: the name, access to the value, the value's identity, or the resources whose future it controlled?

1. The smallest visible boundary

Begin with the smallest case there is: a nested block around an ordinary local variable.

```
{
    let message = String::from("temporary");
    println!("{message}");
}
// message is no longer reachable
```

Scope is the region within which a binding can participate in subsequent computation. Once the closing brace is passed, `message` cannot be read, moved, borrowed, or mutated through that name again, no matter what the program goes on to do. It is worth being precise about what has *not* happened here. The binding has not been mutated; nothing about `message`'s value changed. It has not been moved; no other name received its authority. It has simply stopped being reachable, because the lexical region in which its name was meaningful has ended.

2. Scope closes continuations

Interlude §1's example generalizes: leaving scope removes every continuation that depended on the departing binding. This is best understood as an ending of possibility rather than an ending of the value itself — the value's fate is a separate question, addressed below. What matters first is that the boundary is entirely static. The compiler knows, at the closing brace, exactly which bindings are leaving scope, without running the program. This connects directly to Chapter 5's reachability set $R(v)$: scope's closing brace is one of the places where $R(v)$ shrinks in a way the compiler can verify ahead of time, not one it discovers at runtime. This interlude does not rederive $R(v)$; it starts from Chapter 5's account of reachability and asks what the shrinking itself accomplishes.

3. Release is a consequence, not disappearance

If the binding owned a resource, something else happens at the same boundary: that resource is released. It helps to keep these as two separable facts about the same moment, because beginners often collapse them into one vague idea of a value "going away." The name's reachability ends because the lexical region closed; that is a fact about the program's text. The resource's release happens because ownership ended and nothing else claimed it first; that is a fact about memory and, more generally, about the world outside the program. The first is purely a scoping rule. The second is a consequence Rust attaches to the first.

```
{
    let numbers = vec![1, 2, 3];
    // numbers is reachable, and its allocation exists
}
// numbers is unreachable, and its allocation has been freed
```

The same structure applies beyond heap memory. A file handle, a lock guard, or a temporary directory can each attach their own release behavior — closing a descriptor, releasing a lock, deleting a directory — to the identical lexical boundary.

4. Deterministic destruction

Languages with garbage collection also eventually reclaim unreachable memory, but *eventually* is the operative word: the collector runs on its own schedule, decoupled from the program's lexical structure. Rust's release points are not scheduled; they are determined entirely by where a binding's scope closes in the source text, which means a reader can identify the exact statement after which a resource is guaranteed to be gone. Neither approach is being ranked as universally superior here. The relevant distinction is narrower and more useful: whether the release boundary is predictable from reading the program's structure, or whether it depends on a separate process's own timing. Rust chooses predictability, at the cost of requiring the compiler to track ownership precisely enough to know, statically, where every resource's boundary falls.

5. The `Drop` trait

Rust exposes this boundary directly through the `Drop` trait, which lets a type attach a final consequence to the end of its own ownership.

```
struct Marker(&'static str);

impl Drop for Marker {
    fn drop(&mut self) {
        println!("closing {}", self.0);
    }
}

fn demo() {
    let _first = Marker("first");
    let _second = Marker("second");
    println!("working");
}
// prints: working
// then: closing second
// then: closing first
```

Nothing calls `.drop()` explicitly in `demo`. The compiler inserts the call at the point where each `Marker` leaves scope, silently and automatically, in exactly the same way it would silently free a `String`'s allocation. `Drop::drop` is simply what a type has told the compiler to do at that boundary, in addition to whatever the compiler already does by default.

6. Destruction order preserves history

The output above ends with `second` closing before `first`: reverse declaration order. This is not incidental cleanup trivia. While `second` is being dropped, `first` has not yet been dropped, and remains fully reachable to any code `second`'s destructor might run — including, in more elaborate examples, a destructor that reads a field belonging to a value declared earlier. If destruction ran in declaration order instead, later values could find themselves being torn down while values they might still depend on had already been destroyed. Reverse order guarantees that whatever was still valid a moment ago remains valid while the next thing finishes closing. The same rule applies to nested scopes: an inner scope's contents are fully dropped before the outer scope's closing continues.

7. Early closure with `drop`

Sometimes a resource's release needs to happen before its binding's scope would otherwise end. `std::mem::drop` provides this, not by calling `Drop::drop` directly — user code is not permitted to do that — but by taking ownership of the value and immediately letting *it* go out of scope inside a one-line function body.

```
let guard = mutex.lock().unwrap();
// protected work
drop(guard);
// unprotected work continues here, lock already released
```

The operation being performed is not really "destroy this value on demand." It is "end this continuation region early, so that a different one can begin sooner than the enclosing block would otherwise allow." `drop(guard)` is a deliberate, visible relocation of a boundary that would otherwise sit at the end of the block.

8. RAII as structural coupling

The pattern behind `MutexGuard`, file wrappers, and temporary-directory handles has a name: Resource Acquisition Is Initialization, RAII. The idea is a direct structural coupling between a resource's lifetime and an object's lifetime. While the guard object exists, the invariant it protects — exclusive access, an open handle, an extant directory — is upheld. The moment

the object stops existing, by ordinary scope closure or by an early `drop`, the invariant's protection ends and its cleanup runs. Section 5's `Marker` was a minimal, observable version of exactly this coupling; a real `MutexGuard` does the identical thing with a lock instead of a `println!`.

9. Limits of lexical closure

Static, lexical destruction is powerful but not absolute. `Rc` cycles can keep values mutually reachable forever, so their count never reaches zero and `Drop` never runs. `std::mem::forget` explicitly discards a value without running its destructor at all, by design, for the rare cases where that is exactly what is wanted. Process termination by a hard crash or an external kill skips ordinary unwinding and any destructors along with it. And even when a destructor does run, Rust can guarantee *that* it runs; it cannot guarantee that whatever the destructor attempts — flushing a buffer to a failing disk, closing an already-severed network connection — actually succeeds. Rust's promise is about when a boundary closes in the program's own structure, not about the reliability of everything the world does in response.

Invariant Established

Scope is not merely where a name stops being visible. It is a statically legible boundary at which a set of possible continuations ends and, for anything the boundary owned, a due consequence becomes actual — reliably ordered, but not immune to what lies outside the program's own execution.

Non-overlap note: Chapter 5 establishes who has authority over a value and when that ownership ends. This interlude begins from that premise and studies what the ending of ownership itself does — what closes, in what order, and with what guarantees.